

GITLET

A VERSION CONTROL SYSTEM

Akshay Kumar Sood
kumarakshaysood@gmail.com

GitHub: <https://github.com/akshay-kumar-sood/>

LinkedIn: <https://www.linkedin.com/in/akshay-sood-3692b5297/>

Portfolio: <https://akshaykumarsood.vercel.app/>

POINTERS

I have used file-based pointers. Each pointer is a commit ID stored as a string which points to the commit file inside the .gitlet directory.

Need of Pointers

- To track commit history
- To manage features like log, branching, checkout
- To link commits together

Pointer Structure

HEAD ----> Current Branch Name (Default: main)

Branch / main ----> Latest Commit ID

Commit ----> Parent Commit ID

BASIC WORKFLOW (SINGLY LINKED LIST)

Step 0 (init): HEAD main, Branch null, Commit empty

Step 1 (1st commit): HEAD main, Branch abc123, commit abc123 null

Step 2 (2nd commit): HEAD main, Branch def456, commit def456 abc123 null

Traversal is from latest commit to oldest commit.

FEATURES

FEATURE 0 [INITIAL SETUP]

Commands:

```
cd Gitlet
```

```
javac -d bin src/*.java
```

-d means destination. Converts .java to .class files and stores them in bin.

FEATURE 1 [GIT INIT]

Command:

```
java -cp bin Main init
```

-cp means class path. Runs compiled classes from bin.

Output:

Creates a gitlet folder

- Assigns initial pointers (Step 0)

FEATURE 2 [GIT ADD]

Command:

```
java -cp bin Main add <filename>
```

Use Case:

Copies file from working directory to staging area: Working

Directory -> .gitlet/staging/

FEATURE 3 [GIT COMMIT]

Commands:

```
git commit -m "message"
```

```
java -cp bin Main commit "First Commit"
```

Use Case:

Takes staged files and saves them as a snapshot. Workflow: Working

Directory Staging Commit

Internal Working:

- Get Current Branch main
- Get Parent Commit abc123
- Read Staging Files .gitlet/staging/test.txt

Create Commit Object:

Commit:

message : "message"

parent : "previous commit"

files : snapshot (HashMap)

- Generate ID
- Update branch pointer branch/main def456

Structure:

HEAD main

branches/main def456

commit def456 parent = abc123

commit abc123 parent = null

FEATURE 4 [LOGS]

Command:

```
java -cp bin Main log
```

Use Case: Prints full commit history from latest to oldest.

Core Logic (Traversal):

Read HEAD current branch

Read Branch latest commit

Loop: print commit, move to parent, repeat until null

Uses file-based pointers.

FEATURE 5 [BRANCH]

Command:

```
java -cp bin Main branch <branch-name>
```

Use Case: Creates a new branch pointing to the current commit.

Workflow:

Before: HEAD main, branches/main def456

After: branches/dev def456

The same pointer is copied. Future commits depend on the active branch (HEAD).

CHECKOUT

Command: `java -cp bin Main checkout <branch-name>`

Use Case: Switches branch and restores files.

Steps:

- Load commit branches/dev abc123
- Restore files
- Update HEAD dev

FEATURE 6 [MERGE]

Command:

```
java -cp bin Main merge <branch-name>
```

Use Case: Combines changes from another branch into the current branch (current + target = merged result).

Steps:

- ❖ Get current branch
- ❖ Load commits
- ❖ Apply merge logic

Cases:

Case 1: current = null, target exists (copy file)

Case 2: file exists in both (conflict detected)

Case 3: same file (do nothing)

Important: Does not change HEAD, does not switch branch, Modifies current working files

AUTO MERGE IN REAL GIT

Git uses 3-way merge:

1. Base (common ancestor)
2. Current branch (HEAD)
3. Target branch

FEATURE 6 [AI MODE]

Command:

```
java -cp bin Main ai "your query"
```

Use Case: Converts natural language into Gitlet command using AI.

Example: "save file" -> gitlet commit "save file"

FINAL SUMMARY

- ❖ File-based pointers
- ❖ Singly linked list
- ❖ Branch = pointer
- ❖ HEAD = current branch
- ❖ Commit = snapshot

SERIALIZE AND DESERIALIZE CONCEPT

- ❖ Create Commit Object
- ❖ Commit = { Message, Parent ID, Files (snapshot HashMap), timestamp }
- ❖ Serialize

Commit Object -> Converted to bytes -> stored in file

.gitlet / commits / < commit - id >

```
FileOutputStream fileOut = new FileOutputStream(file);
```

```
ObjectOutputStream out = new ObjectOutputStream(fileOut);
```

```
out.writeObject(obj);
```

Store Pointer

branches/main -> <commit-id>

Later on Need Deserialize

file -> bytes -> Commit object

```
FileInputStream fileIn = new FileInputStream(file);
```

```
ObjectInputStream in = new ObjectInputStream(fileIn);
```

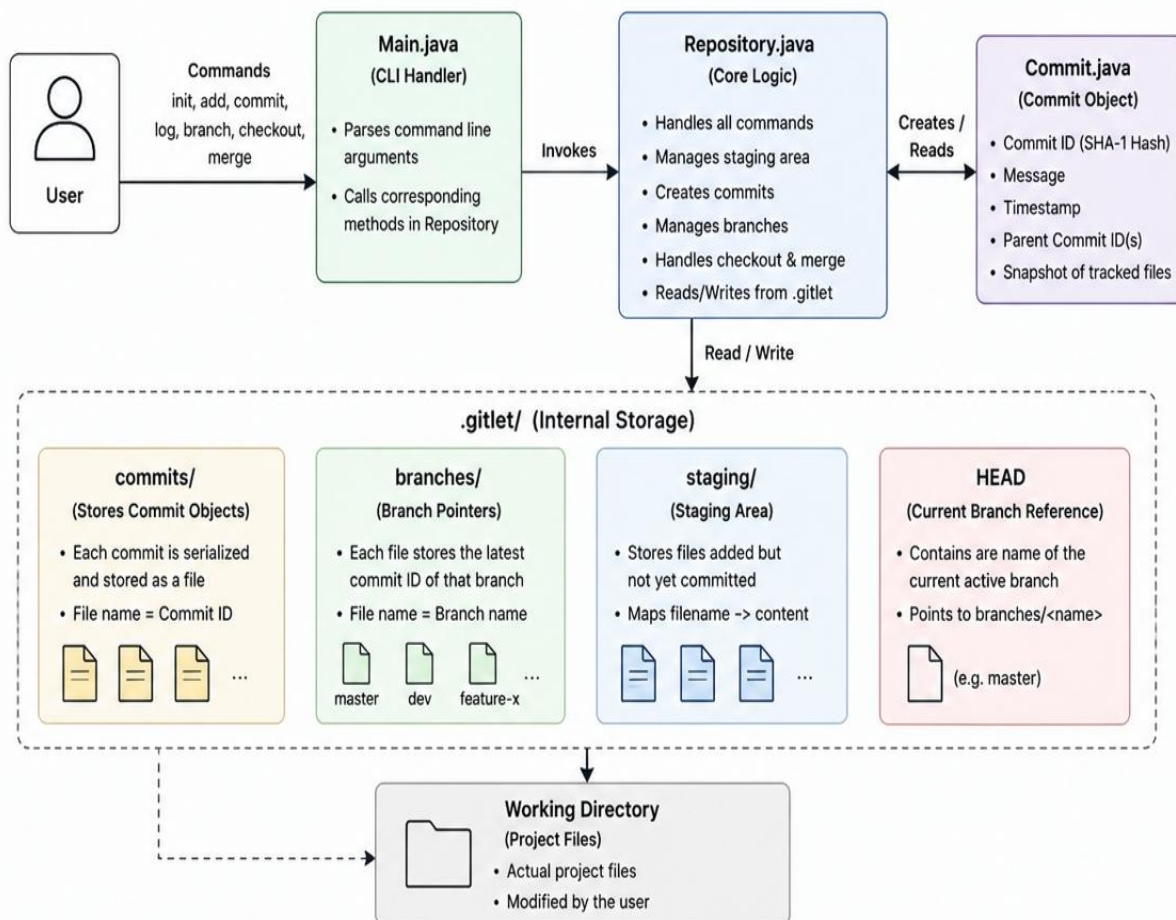
```
T obj = (T) in.readObject();
```

FINAL FLOW

Commit (object) -> Serialize -> Store as file -> Later read file -> Deserialize -> Use object

HLD – HIGH LEVEL DIAGRAM

Gitlet – High Level Design (HLD)



How It Works

1. User runs a command via CLI.
2. Main.java parses the command and calls Repository.
3. Repository performs required operations.
4. Data is read from / written to .gitlet directory.
5. Commits capture snapshots of files and maintain history.

Key Points

- All data is stored in .gitlet
- Commits are immutable snapshots
- Branches are lightweight pointers to commits
- Checkout updates working directory to match a commit

LLD – LOW LEVEL DIAGRAM

Gitlet – Low Level Design (LLD)

